

内部技术评估

DeepSeek V4 Pro / V4 Flash OpenCode vs DeepSeek Harness

从“模型能力”转向“模型 × Harness 的可访问能力”

结论预览

V4 Pro 在 Project2 上表现出显著 Harness 敏感性：OpenCode 正式版均值 92.75，DSH Minimal 均值 97.5，原始两阶段 Anchored 观测均值 98.5；V4 Flash 则在 OpenCode 与 DSH Minimal 都为 92，说明其跨 Harness 泛化更稳。

- 报告目的：为公司内部 Coding Agent / Harness 技术选型提供管理层决策依据。
- 重点对象：DeepSeek V4 Pro 与 DeepSeek V4 Flash；重点比较 OpenCode 与 DeepSeek Harness（DSH）。
- 证据边界：核心跑分来自 Project2 V4.1b 单一冻结工程维护任务，不构成通用模型排名。

日期：2026-08-20 | 内部使用 / 供管理层决策参考

01 EXECUTIVE SUMMARY

结论先行：不要统一 Harness，应按模型特性分流

管理层建议

将 Harness 视为模型能力的一部分，而不是无关紧要的 UI/CLI 外壳。建议采用“模型感知的 Harness 路由”：V4 Pro 建立 DSH 对齐高性能通道；V4 Flash 以 OpenCode 为默认生产通道；同时保留统一的内部评测尺子。

优先级	结论	管理动作
P0	V4 Pro 是 Harness-sensitive：DSH Minimal / Anchored 在本题显著高于常规 OpenCode/DSH Standard。	为 Pro 建立 DSH aligned 专用 lane；OpenCode 保留为 baseline / fallback。
P0	V4 Flash 更 Harness-robust：OpenCode 92，DSH Minimal 92；DSH Standard 反而 90。	Flash 优先 OpenCode，除非需要 DSH 特有 Runtime 能力。
P1	DSH 本身不是“魔法”：DSH Standard 91、PTC 92，与 OpenCode 常规档接近。	不要把“切到 DSH”当成优化；优化对象是 prompt + tool schema + context bootstrap。
P1	Anchored 的高分有强信号，但跨环境/小样本效应仍未定。	生产前在公司 3-5 个不同仓库固定 n 重复 A/B，验证 effect size。
P2	DSH 当前仍为 developer preview，存在兼容性破坏风险。	将 DSH 性能 lane 与生产编排解耦，版本锁定并设回滚。

来源：[S1][S2][S7][S8]

一句话判断

对 V4 Pro，Harness 选择可能相当于“解锁一个更好的策略区域”；对 V4 Flash，Harness 主要改变行为风格，却没有同步改变本题交付能力。

02 PYRAMID LOGIC

金字塔逻辑：一个总判断，三组证据

总判断

Observed Agent Capability 不是裸模型常数，而是 Model × Prompt × Tool Schema × Context Injection × Agent Loop × Runtime 的函数。对 V4 Pro，这个乘法效应在 Project2 上尤其明显。

证据层	核心事实	说明
A. 结果层	Pro：OpenCode 正式均值 92.75；DSH Minimal 97.5；原始 Anchored 98.5。	差距足以进入工程决策，而非 1 分以内噪声。
B. 反事实层	DSH Standard 91、PTC 92；同 WSL/max 下仍低于 Minimal。	排除了“Linux 红利”“官方 DSH 红利”“单一 run_code 入口红利”。
C. 机制层	Minimal 官方测试明确称“exact RL prompt and schemas”；首轮固定 prompt + 两工具。	支持“训练接口对齐 / 首轮策略选择”解释。
D. 反例层	Flash：OpenCode -> Minimal 轨迹巨变，但 Ability 92 -> 92。	证明话语轨迹是 scaffold 指纹，不是高分因果本身。

管理含义

未来模型选型表不应只记录“Model Name”。至少要记录 Model + Provider/Route + Harness + Preset + Thinking Level + Tool Surface + Run Count。

03 EVIDENCE & METHOD

证据口径：为何这些数字可用、又不能被过度外推

- Project2 V4.1b 是一个冻结的多模块 Python + ESP32 工程维护任务，题面、hidden tests、计分规则均冻结；其定位是个人/工程选型 Harness，而非公开通用 benchmark。
- Ability 衡量工程完成度；Ship 叠加安全/可信/发布风险。本报告重点用 Ability 比较“被 Harness 释放出来的交付能力”。
- 正式 Pro OpenCode 为 4 跑；DSH Minimal 2 跑；原始 Anchored 2 跑。Flash OpenCode 与 DSH Standard/Minimal 目前样本更少，因此对 Flash 主要判断“是否存在明显增益”，不做精确稳定性排序。
- DSH 源码审计同时看实验时固定提交与当前 master；当前官方 Minimal 仍保留固定完整 system prompt、两工具与无 compaction 的核心结构。
- 当前社区 anchored-standard 已继续演化，其 README 明确说明原始 98/99/99 跑分早于当前 composition，且独立复验在小样本下尚未确定能力增益。

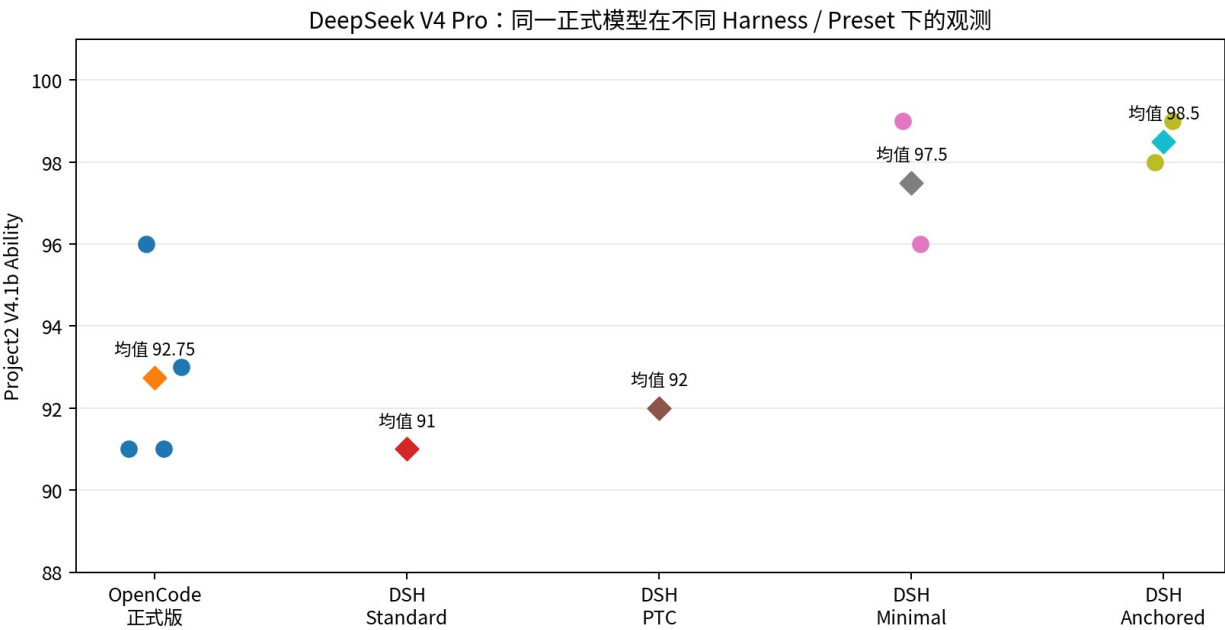
证据等级

A 级：冻结 benchmark 的实际运行结果；B 级：官方源码/测试直接证据；C 级：机制解释与公司建议。报告中不把 C 级推断写成已证明事实。

来源：[S1][S3][S4][S7][S8]

04 DEEPSEEK V4 PRO

V4 Pro：同一正式模型，Harness 可以改变可访问能力档位



配置	单跑 Ability	均值	关键解释
OpenCode 正式版	91 / 96 / 91 / 93	92.75	常规通用 Harness；存在较大轨迹差异。
DSH Standard	91	91	换成官方 DSH 但保持大工具/控制面，并无自然增益。
DSH PTC	92	92	单一 run_code 呈现不等于 RL 对齐。
DSH Minimal	99 / 96	97.5	固定完整 prompt + 两个训练对齐工具；两跑 hidden 均 44/45。
原始 Anchored Standard	98 / 99	98.5	首轮 2 工具，首个工具调用后恢复完整 Standard 目录。

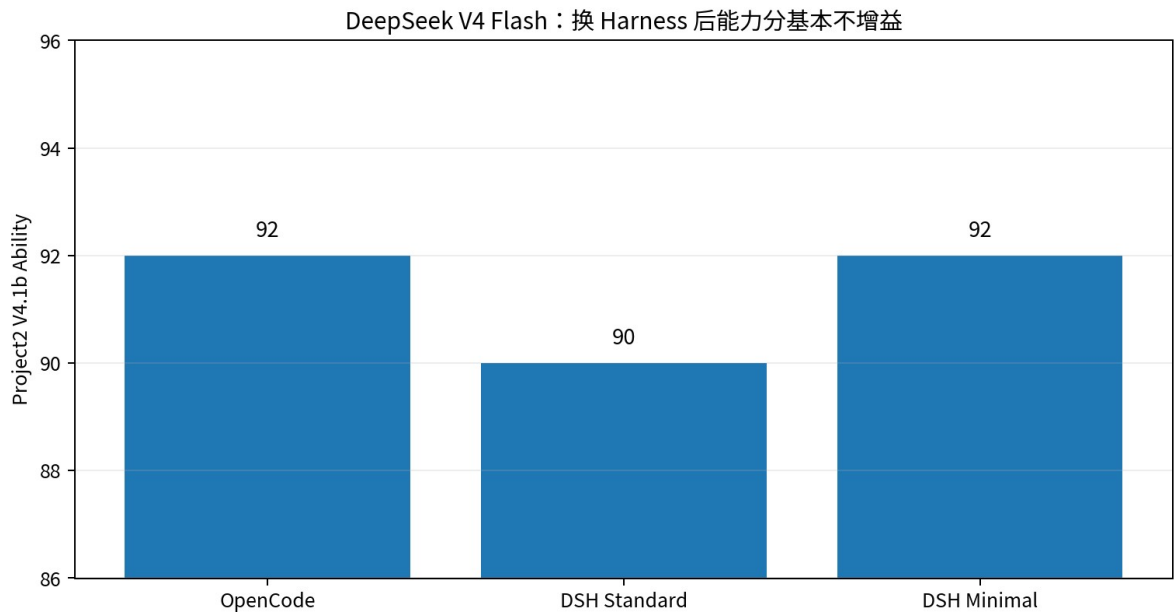
关键解读

“DSH > OpenCode” 是错误结论。正确结论是：对 V4 Pro，DSH 提供了更容易构造训练接口对齐 scaffold 的 Runtime；DSH Standard 本身并没有优于 OpenCode。

来源：[S1][S2]

05 DEEPSEEK V4 FLASH

V4 Flash：Harness 改变了轨迹，但没有带来同等级能力跃迁



配置	Ability	观察
OpenCode	92	通用 Harness 基线。
DSH Standard / Windows	90	官方 Harness 不构成自然增益。
DSH Minimal / WSL / max	92	与 OpenCode 持平。
其他通用 Harness（背景）	Codex 93 / Reasonix 95 / WorkBuddy 93	跨 Harness 的分数区间相对集中，支持其接口泛化更稳的判断。

管理含义

如果目标是“稳定、低运维成本地使用 V4 Flash”，现有证据不足以证明为了性能切换 DSH 值得。OpenCode 更适合作为默认通道；DSH 的价值更多在 Runtime 实验与特定能力组合。

来源：[S1][S2]

06 COUNTERFACTUAL

反事实最重要：DSH Standard 与 PTC 都没有复现 Minimal 增益

对照	控制变量	结果	能排除什么
DSH Standard vs Minimal	同 Ubuntu 24.04、同候选提示、同 max、同构建脚本	91 vs 99/96	排除 Linux 与 “官方 DSH” 本身。
DSH PTC vs Minimal	同环境；PTC wire 上只暴露 run_code	92 vs 99/96	排除 “只要工具入口变成一个” 即可变强。
Anchored vs Standard	Windows；首轮 2 工具后恢复 25 工具	98/99 vs 常规档	排除 “高分必须永远限制在两个工具”。

PTC 的失败模式

PTC 164 次外层 run_code / 187 次内层调用；122 个成功程序只包装 1 次子调用，Promise.all 为 0；另有 10 次缺 description、3 次 TypeScript 执行失败。它增加协议层，却没有形成有效批处理。

- 因此优化目标不是 “工具越少越好”，也不是 “Code Mode 越抽象越好”。
- 更合理的解释是：首轮模型可见分布是否接近后训练/RL 形成的接口分布。
- 如果首轮策略已经被稳定选择，后续可以扩展能力，而不必永久牺牲工具丰富度。

来源：[S1][S2]

07 MECHANISM 1

Minimal 的特殊性：不是“短 Prompt”，而是一整套训练接口分布

组成	DSH Minimal	DSH Standard / 通用 Harness
System prompt	完整固定：You are a helpful software engineer assistant.	更丰富的身份、cwd、工具/运行时指导。
Prompt assembly	complete=true；阻止后续 prompt section 拼接。	由多个 section / instructions 组合。
Runtime context	includeRuntimeContext=false。	可注入 sandbox、approval、workspace instructions 等。
Tool surface	仅 persistent bash/pwsh + str_replace_editor。	约 25 项工具与控制面（读写/搜索/jobs/skills/goals/plan/subagent/workflow/todo/web 等）。
Compaction	无。	Standard 带 compaction / tool result pruning。
官方测试描述	“sends the exact RL prompt and schemas”	没有同样的 exact-RL 表述。

Know-why

Tool Schema 本身就是 Context。模型选择动作的条件不是只有用户问题，而是 Prompt + History + Tool Names + Tool Descriptions + JSON Schemas + Runtime Context。对训练分布敏感的模型，换 Harness 等价于换了一部分“推理输入”。

来源：[S4][S5][S6]

08 MECHANISM 2

Anchored 思路：把“策略选择”和“能力开放”拆成两个阶段

Request #1 Minimal-like Prompt + 2 tools	-> 首次 durable action	Promotion 同一 Session 保持历史	-> 工具目录扩展	Request #2+ 完整/按需工具 继续执行
---	-----------------------------	--	------------------	---

- 原始实验：第一次请求仅暴露 pwsh/read；首个工具调用后恢复完整 Standard 25 项工具，连续得到 98/99。
- 轨迹保持：两次 Anchored 合计 355 个 reasoning 块只有 1 次 “let me”；与 Standard 单轮 208 次明显分离。
- 后续社区版进一步演化为 Progressive Tool Disclosure：先 2 工具，再 resident catalog，再通过 dev_tool_search 按需解锁重型工具。
- 后续还加入 context-gate：首轮抑制 AGENTS/skills/runtime 等自动注入，promotion 后恢复。

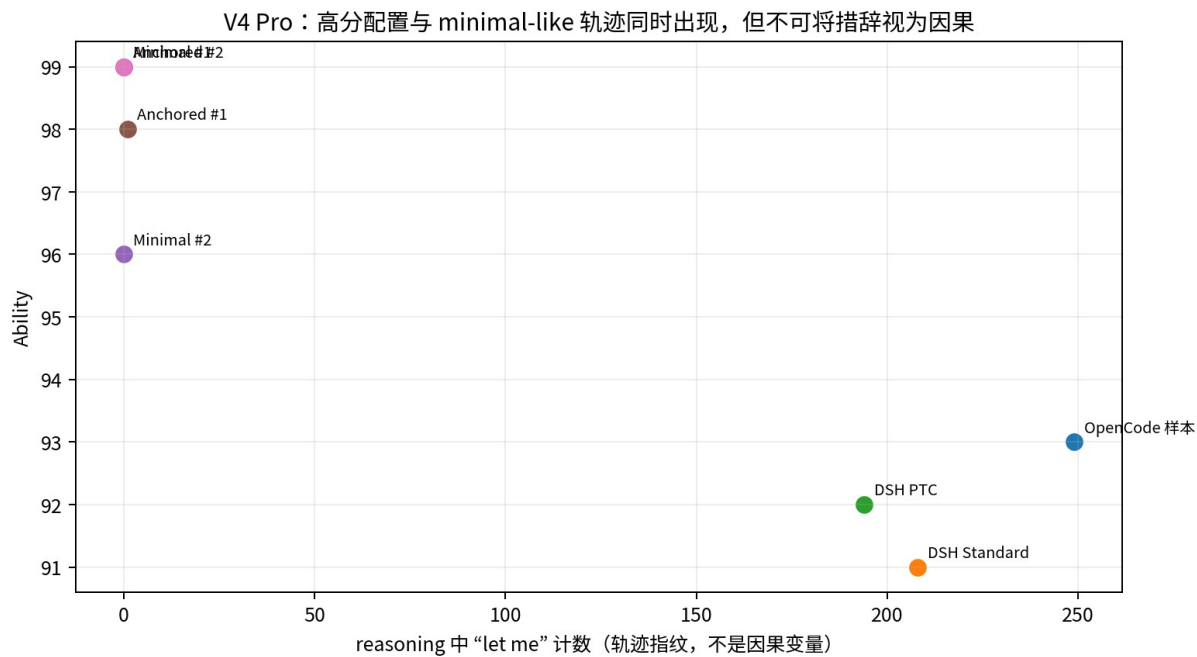
重要限制

当前社区 anchored-standard README 明确：原始 98/99/99 发生在早期 composition；独立复验虽然能稳定复现“轨迹锚定”，但能力提升幅度在小样本下尚未定论。因此应把机制当作“高价值假设”，而非可直接承诺的 SLA。

来源：[S2][S7]

09 BEHAVIORAL EVIDENCE

V4 Pro 的轨迹证据：高分与 minimal-like 行为同步出现



样本	Ability	reasoning p50	let me	阶段回复	工具调用
OpenCode 正式样本	93	973	249	37	274
DSH Standard	91	437	208	55	189
DSH PTC	92	550	194	33	164 外层
DSH Minimal #1	99	235	0	1	194
DSH Minimal #2	96	239	0	1	171
Anchored #1	98	111	1	1	242
Anchored #2	99	144	0	1	201

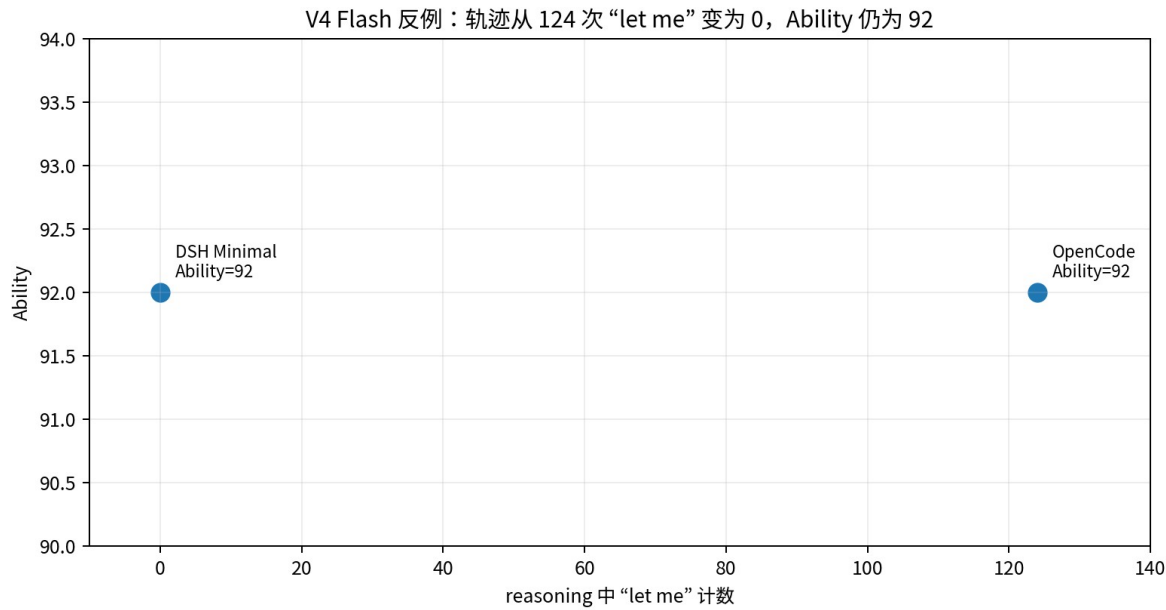
不要误读

“let me=0” “We need” 只是 scaffold 是否生效的轨迹指纹，不是能力因果变量，也不能用于鉴定模型后端身份。

来源：[S2]

10 FLASH COUNTEREXAMPLE

V4 Flash 是关键反例：风格可以完全变，能力却不变



指标	OpenCode Flash	DSH Minimal Flash	变化
Ability	92	92	0
reasoning p50 字符	365	128	显著缩短
we	5	209	显著上升
let me	124	0	完全消失
阶段回复	47	1	接近只保留最终回复
工具调用	149	178	没有因更短轨迹而明显减少

结论

Flash 的后训练对 Prompt / Tool 呈现变化更鲁棒：Scaffold 可以强烈改变“怎么说、怎么组织推理”，但本题的最终工程能力没有同步跃迁。

对选型的意义

V4 Pro 与 V4 Flash 不应共用同一 Harness 策略。把模型当成同一“DeepSeek V4 系列”做统一接入，会损失 Pro 的潜在上限，也会给 Flash 增加不必要的 Runtime 复杂度。

来源：[S2]

11 OPENCODE VS DSH

架构层面的差别：通用 Coding Agent 产品 vs 可重组 Agent Runtime

维度	OpenCode	DeepSeek Harness (DSH)	对本项目的影响
第一目标	通用开源 Coding Agent；build / plan 等用户态 agent。	Everything is a Plugin；Harness/Runtime 可组合。	OpenCode 更适合统一生产体验；DSH 更适合 model-specific scaffold。
Tool Surface	成熟的通用工具/权限/插件/MCP 生态。	Tool registry、schema assembly、execution pipeline 都是可替换 seam。	DSH 更容易做动态工具面与首轮 gating。
Prompt / Context	以产品配置和 agent instructions 为中心。	system-prompt service 可被 preset 组合/覆盖；agent/pre-step 可拦截。	更容易严格控制首轮模型可见分布。
Agent Loop	产品实现的核心机制。	agent-loop 本身也是插件。	适合实验不同 lifecycle、turn/step 行为。
Session	面向用户会话与客户端体验。	append-only SessionEvent；“Model-visible means logged”。	更适合 resume-safe 的 phase / promotion 状态。
稳定性	产品化程度高、跨模型定位明确。	官方仍标记 developer preview，存在 breaking changes。	DSH 性能 lane 需要版本锁定与回滚。

判断

OpenCode 的价值是“统一 ABI”；DSH 的价值是“Agent Microkernel”。对接口鲁棒的模型，统一 ABI 更经济；对接口敏感但上限高的模型，Microkernel 的可组合性更有价值。

来源：[S3][S4][S9]

12 DECISION MATRIX

公司选型建议：按“模型 × 场景”而不是按“一个 Harness 打天下”

模型 / 场景	默认选择	备选 / 性能通道	理由	治理要求
V4 Pro / 高价值复杂改造	DSH aligned lane	OpenCode fallback	本题观察到 5-6 分以上的均值差；Pro 明显 Harness-sensitive。	锁版本；记录 preset/tool surface；跨 repo A/B。
V4 Pro / 日常低风险任务	OpenCode 或 DSH Standard baseline	DSH aligned 按任务升级	通用体验和运维简单性优先；不必每个任务都付出专用 Runtime 成本。	自动升级条件；失败可回退。
V4 Flash / 默认开发	OpenCode	DSH Minimal 仅实验	现有证据 OpenCode=Minimal=92；DSH Standard 90。	保持统一工具/权限治理。
多模型平台	OpenCode 作为统一前台/基线	DSH 作为模型特化执行后端	兼顾开发者体验与模型特定优化。	统一证据链、权限、安全策略。

不建议

不建议直接把当前社区 anchored-standard 仓库作为公司生产标准组件：其当前实现已不同于原始跑分版本，且维护状态/复验结论不足以支撑生产 SLA。应吸收“首轮对齐 + 动态能力开放”的机制，自建最小可控实现。

13 TARGET ARCHITECTURE

建议目标架构：Model-aware Harness Router + 统一 Eval / Safety Plane

层	建议组件	职责
入口层	统一 IDE / CLI / API	对开发者隐藏底层 Harness 差异。
Harness Router	model_id + task_class + risk_level -> route	V4 Pro 高价值任务进入 DSH aligned；Flash 默认 OpenCode。
Execution Lane A	OpenCode baseline	通用、多模型、稳定工具与权限体验。
Execution Lane B	DSH aligned experimental/performance	严格控制首轮 prompt / tools / context，promotion 后按需开放能力。
Policy Plane	统一 allow/ask/deny、sandbox、secret policy	不因 Harness 不同而降低公司安全标准。
Evidence Plane	diff、test、build、tool trace、cost、run metadata	确保任何性能结论可复算。
Eval Plane	固定公司仓库集 + hidden tests + multi-run	决定何时允许某模型/某 preset 进入生产。

核心工程原则

优化 Harness 时，必须把安全策略与性能 scaffold 解耦。不要因为 Minimal 的本地文件系统/无 sandbox 配置在实验中得分高，就复制成公司生产安全策略。性能对齐可以保留，权限与隔离必须重新挂回公司 Policy Plane。

14 INTERNAL VALIDATION PLAN

上线前验证：用公司自己的仓库验证“是否真有收益”

1. 建立 3-5 个结构差异明显的内部 repo：后端服务、前端/TypeScript、数据/ML、基础设施、嵌入式或系统代码（按公司实际业务裁剪）。
2. 每个 repo 固定 broken seed / task spec / public checks / hidden checks；评分区与候选工作区严格隔离。
3. 对 V4 Pro 至少比较 OpenCode baseline、DSH Standard、DSH Minimal/公司版 Anchored；同 provider、同模型版本、同 reasoning 档、同任务提示。
4. 每个正式配置固定相同 run 数；建议 $n \geq 5$ 用于内部选型，至少报告 mean / median / worst / range，不混用不同 n 的“稳定性”结论。
5. 除 Ability 外，强制记录 Ship/安全 blocker、时长、token/context、工具调用数、失败恢复次数、build 真实性与报告一致性。
6. 只有当 Pro 的 DSH lane 在多数 repo 保持稳定正收益且无安全/运维倒退，才进入默认路由；否则保留为按需性能模式。

成功门槛建议（内部推断）

建议设为：跨 repo 平均 Ability 至少 +3，且 worst-run 不下降；安全/Ship blocker 不增加；总成本和 P95 时长在业务可接受范围。该门槛不是来源中的既有规则，而是面向公司上线的保守建议。

15 RISKS & LIMITATIONS

需要领导明确知道的 6 个边界

风险	影响	处理方式
单一 benchmark	Project2 上的 +5~6 分均值差不等于所有工程都成立。	跨仓库复验。
样本数有限	Minimal/Anchored 的 n 较小；Flash 关键对照更少。	统一 n，多次正式运行。
Anchored 版本漂移	当前社区实现已不同于原始 98/99 版本。	不要直接复用跑分数字为当前代码背书。
独立复验效应未定	当前 README 报告 anchoring 可复现，但能力 gap 小样本 CI 跨 0。	将其作为研究假设，不承诺固定 uplift。
DSH developer preview	可能发生 breaking changes。	版本锁定、兼容性测试、回滚。
安全配置不可照搬	Minimal 实验配置的本地 FS / 无 sandbox 是对齐条件的一部分。	公司自建 Policy Plane，性能 scaffold 与安全隔离分层。

最危险的误判

把“V4 Pro + DSH aligned 在一个 benchmark 上 98/99”直接翻译成“DSH 生产必然提升 6 分”——这是当前证据不支持的。正确说法是：发现了一个高价值、可机制解释、值得公司内部复验的性能通道。

来源：[S1][S3][S7]

16 FINAL RECOMMENDATION

最终建议：双轨 Harness，而不是二选一

建议决策

批准 “OpenCode Baseline + DSH Pro Performance Lane” 的双轨技术验证。V4 Flash 继续以 OpenCode 为默认；V4 Pro 在高价值复杂任务中优先验证 DSH aligned。所有 lane 共用统一安全、证据与评测平面。

未来 30 天	负责人输出	验收
第 1 周	冻结 OpenCode baseline 与 DSH 版本；实现最小 Pro aligned preset；建立 metadata 规范。	每次 run 可准确复现 model/provider/harness/preset/tools。
第 2 周	选 3 个内部 repo，构造可重复任务与 hidden eval。	不同结构、不同工具需求，不复用 Project2。
第 3 周	固定 n 跑 Pro：OpenCode / DSH Standard / DSH aligned；Flash 做 OpenCode / DSH minimal 对照。	形成分数、成本、时长、安全、稳定性矩阵。
第 4 周	形成 routing policy 与上线/放弃结论。	有明确 effect size、worst-run、回滚方案。

决策原则

若公司只想要一个通用 Coding Agent 前台，选 OpenCode；若公司希望榨出 V4 Pro 的潜在上限，就需要额外拥有 Harness Engineering 能力。两者不是替代关系，而是 “统一产品层 + 模型特化执行层” 的关系。

APPENDIX

参考来源与链接

核心事实均来自公开 GitHub 仓库；时间敏感内容按 2026-08-20 口径。

[S1] xiaobright/modeltest - DeepSeek V4 Pro 正式版 harness 对照分析

https://github.com/xiaobright/modeltest/blob/main/docs/v4.1/DEEPSEEK_V4_PRO_HARNESS_ANALYSIS_20260814.md

[S2] xiaobright/modeltest - DeepSeek V4 Pro 轨迹风格与 PTC 对照分析

https://github.com/xiaobright/modeltest/blob/main/docs/v4.1/DEEPSEEK_V4_TRAJECTORY_ANALYSIS_20260814.md

[S3] deepseek-ai/deepseek-harness - README（developer preview / Everything is a Plugin）

<https://github.com/deepseek-ai/deepseek-harness/blob/master/README.md>

[S4] deepseek-ai/deepseek-harness - Architecture

<https://github.com/deepseek-ai/deepseek-harness/blob/master/docs/architecture.md>

[S5] deepseek-ai/deepseek-harness - Minimal preset

<https://github.com/deepseek-ai/deepseek-harness/blob/master/apps/cli/config/agent-presets/minimal/agent.cordis.yml>

[S6] deepseek-ai/deepseek-harness - Minimal snapshot test: exact RL prompt and schemas

<https://github.com/deepseek-ai/deepseek-harness/blob/master/apps/web/tests/minimal-preset.snapshot.ts>

[S7] xiaobright/dsh-anchored-standard - 当前 README（机制、版本漂移、独立复验限制）

<https://github.com/xiaobright/dsh-anchored-standard/blob/main/README.md>

[S8] xiaobright/modeltest - V4.1b scoreboard

https://github.com/xiaobright/modeltest/blob/main/evaluator/reports/v4.1b_scoreboard.md

[S9] anomalyco/opencode - 官方仓库 README

<https://github.com/anomalyco/opencode/blob/dev/README.md>

引用与解释边界

本报告中的跑分、源码结构、测试描述均以以上来源为依据；“公司建议 / 上线门槛 / 目标架构”属于基于这些证据的工程推断，不是原仓库作者的官方结论。